VIETNAM GENERAL CONFEDERATION OF LABOUR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY

FINAL REPORT
DIGITAL IMAGE PROCESSING

Instructor: PhD Trinh Hung Cuong
Student: Tran Tuan Kiet – 521H0461
Nguyen Van Truong – 521H0324
Group: 2

HO CHI MINH CITY, YEAR 2024

VIETNAM GENERAL CONFEDERATION OF LABOUR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY

FINAL REPORT
DIGITAL IMAGE PROCESSING

Instructor: **PhD Trinh Hung Cuong**

Student: **Tran Tuan Kiet – 521H0461**

Nguyen Van Truong – 521H0324

Group: **2**

HO CHI MINH CITY, YEAR 2024

ACKNOWLEDGEMENT

We sincerely thank Mr. Trinh Hung Cuong for teaching us the Information Security course with great enthusiasm. We want to express our deep appreciation for the dedication and professional knowledge that you shared with us. Through your classes, we gained a better understanding of the fundamental aspects of Information Security, thanks to your detailed explanations and practical applications. You helped us grasp the knowledge and apply it effectively. Finally, we extend our heartfelt gratitude to Mr. Trinh Hung Cuong for your commitment and invaluable support throughout our learning journey in this course. The skills and knowledge we acquired will continue to impact our future development. We sincerely thank you and wish you health, success, and happiness.

Ho Chi Minh City, April 15th, 2025

Authors:

Tran Tuan Kiet

Nguyen Van Cuong

DECLARATION OF AUTHORSHIP

We hereby declare that this thesis was carried out by ourselves under the guidance and supervision of Mr. Trinh Hung Cuong; and that the work and the results contained in it are original and have not been submitted anywhere for any previous purposes. The data and figures presented in this thesis are for analysis, comments, and evaluations from various resources by our own work and have been duly acknowledged in the reference part.

In addition, other comments, reviews, and data used by other authors and organizations have been acknowledged and explicitly cited.

We will take full responsibility for any fraud detected in my thesis. Ton Duc Thang University is unrelated to any copyright infringement caused by my work (if any).

Ho Chi Minh City, April 15th, 2025

Authors:

Tran Tuan Kiet

Nguyen Van Cuong

DIGITAL IMAGE PROCESSING
ABSTRACT

This project presents the development and implementation of a digital image processing system using Python, with a primary focus on enhancing, analyzing, and extracting meaningful information from images. Through the integration of multiple image processing techniques, the system aims to address real-world challenges in image interpretation and analysis.

The image processing pipeline includes stages such as image acquisition, grayscale conversion, noise reduction, contrast enhancement, edge detection, and morphological operations. These processes enable the extraction of key visual features, segmentation of regions of interest, and preparation for higher-level analysis or classification.

Open-source Python libraries, including OpenCV, are utilized to perform pixel-level manipulations and apply spatial filters. Depending on the application, the system may also implement advanced methods such as thresholding (global/adaptive), contour detection, and Hough Transform for shape recognition.

Special emphasis is placed on improving image quality, reducing artifacts, and ensuring robustness under varying lighting or environmental conditions.

This project demonstrates the practical application of digital image processing concepts and tools to solve visual problems efficiently. It serves as a foundation for future work in fields such as computer vision, medical imaging, remote sensing, and intelligent surveillance.

TABLE OF CONTENTS

CHAPTER I: METHODLOGY OF SOLVING TASKS

1. Video-Based Traffic Sign Detection Using Color Masking and Contour Analysis

1.1. Overview:

This task focuses on detecting traffic signs in a video stream based on their color characteristics (primarily red and blue). The system processes each frame of the video using color space conversion, histogram equalization, threshold-based segmentation, morphological operations, and contour-based bounding box detection. The entire process is implemented using Python with OpenCV and NumPy libraries.

This task involves basic digital image processing techniques, including:

- Color filtering in HSV color space
- Morphological operations to clean noise
- Contour detection to find object candidates
- Template matching to classify signs
- Video frame annotation and saving

1.2. Background Knowledge and Applied Methods:

1.2.1 Color Filtering (HSV Space)

- RGB is not ideal for object segmentation due to its sensitivity to lighting. HSV (Hue, Saturation, Value) separates color (Hue) from intensity (Value), making it more robust.
- The code uses HSV ranges to filter **red**, **blue**, and **yellow** – typical colors used in traffic signs.

1.2.2 Morphological Operations

- These operations remove noise from binary images:

- cv2.morphologyEx(..., cv2.MORPH_OPEN, ...) removes small white dots (noise).
- cv2.morphologyEx(..., cv2.MORPH_CLOSE, ...) fills small black holes inside white areas.

1.2.3 Contour Detection

- Contours are boundaries of objects. OpenCV's cv2.findContours helps isolate potential sign regions after thresholding.

1.2.4 Template Matching

- This is a basic image classification method.
- Each detected Region of Interest (ROI) is resized and compared to preloaded template images using cv2.matchTemplate.
- The best-matched template determines the sign's label.

1.2.5 Video Handling

- OpenCV is used for reading (cv2.VideoCapture) and writing (cv2.VideoWriter) frames to/from video files.
- Frames are annotated with rectangles and labels using cv2.rectangle() and cv2.putText().

1.3. Main Programming Steps

Initialize Templates: Load reference sign images from a folder into a dictionary.

Filter Colors: Convert video frame to HSV and create a binary mask for traffic sign colors.

Find Contours: Use the mask to extract sign-like regions.

Verify and Classify: Filter by size and aspect ratio, use template matching to classify signs.

Annotate Frame: Draw rectangles and labels.

Save Output Video: Write annotated frames into a new video file.

1.4. Code Explanation:

1.4.1. Global Variables:

```
# Biến toàn cục lưu các ảnh mẫu
TEMPLATE_SIGNS = {}

# Bản đồ ánh xạ tên biển báo
SIGN_LABELS = {
```

TEMPLATE_SIGNS: A global dictionary that stores loaded traffic sign templates for later matching.

SIGN_LABELS: A dictionary that maps template file names to human-readable sign labels (e.g., "noleft" → "No left turn").

1.4.2. Initialize Template:

Purpose:

This function loads traffic sign template images from a given directory and stores them in a global dictionary named TEMPLATE_SIGNS. These templates will later be used to identify signs in video frames through image matching.

```
Tabnine | Edit | Test | Explain | Document
def initialize_templates(directory='sign_templates'):
    """Nạp các ảnh mẫu biển báo từ thư mục."""
    for file in os.listdir(directory):
        if file.lower().endswith(('.jpg', '.png')):
            label = os.path.splitext(file)[0]
            path = os.path.join(directory, file)
            img = cv2.imread(path)
            if img is not None:
                TEMPLATE_SIGNS[label] = img
```

def compare_roi_with_template(roi, template_img, size=(100, 100)):
Defines a function that compares a Region of Interest (ROI) from a video frame with a template image. The images are resized to a common size for consistent matching.

roi_resized = cv2.resize(roi, size)
Resizes the input ROI to a standard dimension (default is 100x100 pixels) to normalize the comparison process.

template_resized = cv2.resize(template_img, size)
Resizes the template image to the same size as the ROI to ensure both have matching dimensions during template matching.

roi_gray = cv2.cvtColor(roi_resized, cv2.COLOR_BGR2GRAY)
Converts the resized ROI image from BGR to grayscale, which simplifies the data and speeds up matching.

template_gray = cv2.cvtColor(template_resized, cv2.COLOR_BGR2GRAY)
Also converts the resized template image to grayscale to ensure consistency in comparison.

result = cv2.matchTemplate(roi_gray, template_gray, cv2.TM_CCOEFF_NORMED)
Performs normalized cross-correlation between the grayscale ROI and template to determine similarity. Higher values mean better matches.

return np.max(result)
Returns the maximum match score from the result matrix, which represents the highest degree of similarity between the ROI and the template.

1.4.3. Comparing ROI with Template Using Normalized Cross-Correlation:

Purpose:

This function compares a region of interest (ROI) extracted from the input video frame with a reference traffic sign template image. It standardizes both images to the same size, converts them to grayscale, and uses OpenCV's template matching method to calculate a similarity score.

```
Tabnine | Edit | Test | Explain | Document
def compare_roi_with_template(roi, template_img, size=(100, 100)):
    """So khớp ảnh ROI với template theo kích thước chuẩn."""
    roi_resized = cv2.resize(roi, size)
    template_resized = cv2.resize(template_img, size)

    roi_gray = cv2.cvtColor(roi_resized, cv2.COLOR_BGR2GRAY)
    template_gray = cv2.cvtColor(template_resized, cv2.COLOR_BGR2GRAY)
                                  (variable) template_gray: Any

    result = cv2.matchTemplate(roi_gray, template_gray, cv2.TM_CCOEFF_NORMED)
    return np.max(result)
```

roi_resized = cv2.resize(roi, size)
Resizes the ROI image to a fixed dimension (default 100x100 pixels) to ensure uniform input size for the comparison process.

template_resized = cv2.resize(template_img, size)
Resizes the template image to match the dimensions of the ROI, maintaining consistency for valid comparison.

roi_gray = cv2.cvtColor(roi_resized, cv2.COLOR_BGR2GRAY)
Converts the resized ROI to grayscale to simplify the data and reduce computational cost, since color is not needed for template matching.

template_gray = cv2.cvtColor(template_resized, cv2.COLOR_BGR2GRAY)
Converts the template image to grayscale to ensure it is in the same format as the ROI during comparison.

result = cv2.matchTemplate(roi_gray, template_gray, cv2.TM_CCOEFF_NORMED)
Performs template matching using normalized cross-correlation. It evaluates how well the grayscale template matches each location in the ROI.

return np.max(result)
Returns the highest matching score found in the result matrix. This score quantifies how similar the ROI is to the template — the closer the value is to 1, the better the match.

1.4.4. Filtering Sign Regions by Color and Extracting Contours:

Purpose:

This section of the function identifies potential traffic sign areas in a video frame by filtering specific colors (red, blue, yellow) and extracting clean contours using image morphology.

```
Tabnine | Edit | Test | Explain | Document
def identify_signs_in_frame(frame):
    """Nhận diện biển báo giao thông trong 1 khung hình."""
    hsv_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

    color_ranges = {
        "Red": (np.array([150, 50, 50]), np.array([180, 255, 255])),
        "Blue": (np.array([90, 120, 100]), np.array([130, 255, 255])),
        "Yellow": (np.array([0, 120, 0]), np.array([30, 255, 255]))
    }

    # Tạo mask kết hợp từ các màu
    mask_combined = np.zeros(hsv_frame.shape[:2], dtype=np.uint8)
    for lower, upper in color_ranges.values():
        mask_combined = cv2.bitwise_or(mask_combined, cv2.inRange(hsv_frame, lower, upper))

    # Làm sạch nhiễu
    kernel = np.ones((3, 3), np.uint8)
    mask_cleaned = cv2.morphologyEx(mask_combined, cv2.MORPH_OPEN, kernel)
    mask_cleaned = cv2.morphologyEx(mask_cleaned, cv2.MORPH_CLOSE, kernel)

    contours, _ = cv2.findContours(mask_cleaned, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    detected = []
```

hsv_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
Converts the input frame from BGR to HSV color space, which is more robust for color-based segmentation.

color_ranges = { ... }
Defines HSV value ranges for three traffic-sign-relevant colors: red, blue, and yellow. Each color is associated with a lower and upper bound.

mask_combined = np.zeros(hsv_frame.shape[:2], dtype=np.uint8)
Creates an empty binary mask with the same height and width as the frame to accumulate color detections.

for lower, upper in color_ranges.values():
Starts a loop over the defined color ranges.

mask_combined = cv2.bitwise_or(mask_combined, cv2.inRange(hsv_frame, lower, upper))
Applies cv2.inRange() to create a binary mask for the current color, then combines it with the main mask using a bitwise OR operation.

kernel = np.ones((3, 3), np.uint8)
Defines a 3x3 structuring element used for morphological operations to remove noise.

mask_cleaned = cv2.morphologyEx(mask_combined, cv2.MORPH_OPEN, kernel)
Applies an opening operation to eliminate small white noise (foreground specks).

mask_cleaned = cv2.morphologyEx(mask_cleaned, cv2.MORPH_CLOSE, kernel)
Applies a closing operation to fill small black holes inside white regions.

contours, _ = cv2.findContours(mask_cleaned, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
Finds the external contours from the cleaned binary mask, which may represent traffic signs.

detected = []
Initializes an empty list to store detected traffic signs.

1.4.5. Analyzing Contours, Matching Templates, and Assigning Labels:

Purpose:

This part filters valid traffic sign shapes, identifies the dominant color, compares them to templates, and appends labeled results to the detection list.

```
for cnt in contours:
    if cv2.contourArea(cnt) > 500:
        x, y, w, h = cv2.boundingRect(cnt)
        aspect = w / float(h)
        if 0.8 <= aspect <= 1.5:
            roi = frame[y:y+h, x:x+w]
            roi_hsv = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV)

            pixel_counts = {
                color: cv2.countNonZero(cv2.inRange(roi_hsv, *range_))
                for color, range_ in color_ranges.items()
            }

            dominant_color = max(pixel_counts, key=pixel_counts.get)

            match_name = None
            top_score = 0

            for name, tmpl in TEMPLATE_SIGNS.items():
                score = compare_roi_with_template(roi, tmpl)
                if score > 0.55 and score > top_score:
                    top_score = score
                    match_name = name

            if match_name:
                label = SIGN_LABELS.get(match_name, match_name)
                detected.append((x, y, w, h, f"{label} ({dominant_color})"))

return detected
```

for cnt in contours:
Starts looping through each detected contour.

if cv2.contourArea(cnt) > 500:
Filters out small contours (noise) by only processing those with area greater than 500 pixels.

x, y, w, h = cv2.boundingRect(cnt)
Calculates the bounding rectangle for the current contour, providing the top-left corner and dimensions.

aspect = w / float(h)
Computes the aspect ratio of the bounding box to verify that it’s approximately square or rectangular like a typical sign.

if 0.8 <= aspect <= 1.5:
Accepts only bounding boxes with a reasonable aspect ratio to eliminate unlikely shapes.

roi = frame[y:y+h, x:x+w]
Extracts the Region of Interest (ROI) — the portion of the frame where the sign might be.

roi_hsv = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV)
Converts the ROI to HSV for accurate color analysis.

pixel_counts = { color: cv2.countNonZero(cv2.inRange(roi_hsv, *range_)) for color, range_ in color_ranges.items() }
Counts how many pixels in the ROI match each predefined color range, creating a dictionary of color intensities.

dominant_color = max(pixel_counts, key=pixel_counts.get)
Finds the color with the highest number of matching pixels — the dominant color of the sign.

match_name = None
Initializes a variable to hold the matched template name (if any).

top_score = 0
Initializes the best match score found so far.

for name, tmpl in TEMPLATE_SIGNS.items():
Loops over all template images loaded earlier.

score = compare_roi_with_template(roi, tmpl)
Compares the current ROI with the template using the template matching function.

if score > 0.55 and score > top_score:
Checks if the match is confident (above 0.55) and is the best match so far.

top_score = score
Updates the best score.

match_name = name
Updates the matched template name.

if match_name:
If a valid template match is found, proceeds to label it.

label = SIGN_LABELS.get(match_name, match_name)
Converts the internal template name to a readable label using a dictionary, or defaults to the template name.

detected.append((x, y, w, h, f"{label} ({dominant_color})"))
Appends the result to the detected list as a tuple containing the bounding box and the sign label with dominant color.

return detected
Returns the list of detected signs with their bounding box coordinates and labels.

1.4.6. Initializing Video I/O and Preparing Output Writer:

Purpose:

This part of the function prepares the input video for processing by reading its properties and initializing an output video writer to store the annotated result.

```
Tabnine | Edit | Test | Explain | Document
def annotate_and_save_video(input_video_path, output_name='output_video1.mp4', student_id='521H0324_521H0461'):
    """Xử lý video: nhận diện và gán nhãn biển báo, sau đó xuất ra video mới."""
    initialize_templates()

    cap = cv2.VideoCapture(input_video_path)
    ret, frame = cap.read()
    if not ret:
        print("Không đọc được video.")
        return

    height, width = frame.shape[:2]
    fps = int(cap.get(cv2.CAP_PROP_FPS)) or 25

    if os.path.exists(output_name):
        os.remove(output_name)

    writer = cv2.VideoWriter(output_name,
                             cv2.VideoWriter_fourcc(*'avc1'),
                             fps, (width, height))
```

initialize_templates()
Loads all template images from the predefined folder into memory. These templates are used later to recognize traffic signs in video frames.

cap = cv2.VideoCapture(input_video_path)
Opens the input video file at the given path and creates a VideoCapture object to read frames sequentially.

ret, frame = cap.read()
Reads the first frame of the video. The variable ret will be False if reading fails.

if not ret:
Checks whether the frame was successfully read.

print("Không đọc được video.")
Prints an error message in case the video cannot be read.

return
Exits the function early if the video file is unreadable, avoiding errors in later steps.

height, width = frame.shape[:2]
Extracts the height and width of the first frame, which defines the resolution of the video.

fps = int(cap.get(cv2.CAP_PROP_FPS)) or 25
Retrieves the frame rate (frames per second) of the input video. If it returns 0 or fails, the code defaults to 25 fps.

if os.path.exists(output_name):
Checks whether a file with the intended output name already exists.

os.remove(output_name)
Deletes the existing output file to avoid conflicts when writing the new video.

writer = cv2.VideoWriter(output_name, cv2.VideoWriter_fourcc(*'avc1'), fps, (width, height))
Initializes a VideoWriter object that creates a new video file with the specified name, codec (avc1), frame rate, and resolution. This object is used to write the processed frames with annotations.

1.4.7. Annotating Detected Signs on a Video Frame:

Purpose:
This function takes a single video frame, detects traffic signs in it, and overlays rectangles and labels for each detected sign, including a student ID for identification purposes.

```
def annotate_frame(frame):
    signs = identify_signs_in_frame(frame)
    for (x, y, w, h, label) in signs:
        cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
        cv2.putText(frame, label, (x, y-10), cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0, 0, 255), 2)
    cv2.putText(frame, f"Student ID: {student_id}", (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
    return frame
```

signs = identify_signs_in_frame(frame)
Calls the detection function to identify all traffic signs in the given frame. The result is a list of bounding boxes and labels for each detected sign.

for (x, y, w, h, label) in signs:
Loops through each detected sign, where x, y, w, and h define the bounding box, and label is the description of the sign.

cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
Draws a green rectangle around the detected sign on the frame. The thickness of the rectangle border is 2 pixels.

cv2.putText(frame, label, (x, y-10), cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0, 0, 255), 2)
Places the traffic sign label (e.g., "No left turn") just above the rectangle using red text with font size 0.9.

cv2.putText(frame, f"Student ID: {student_id}", (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
Displays the student ID in green text at the top-left corner of the frame for identification and tracking.

return frame
Returns the modified frame with all annotations applied so it can be displayed or saved to output video.

1.4.8. Processing and Saving Annotated Frames in a Video Loop:

Purpose:
This part of the function continuously reads each frame of the video, applies traffic sign annotations, displays the result, writes it to the output file, and allows the user to exit by pressing the 'q' key.

```
    frame = annotate_frame(frame)
    cv2.imshow("Video Output", frame)
    writer.write(frame)

    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break
        frame = annotate_frame(frame)
        writer.write(frame)
        cv2.imshow("Video Output", frame)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

    cap.release()
    writer.release()
    cv2.destroyAllWindows()
```

frame = annotate_frame(frame)
Applies annotations (bounding boxes, labels, student ID) to the first frame before entering the loop.

cv2.imshow("Video Output", frame)
Displays the annotated first frame in a window titled “Video Output”.

writer.write(frame)
Writes the annotated first frame to the output video file.

while cap.isOpened():
Starts a loop that continues as long as the video file is open and frames can still be read.

ret, frame = cap.read()
Reads the next frame from the input video. If no frame is left or an error occurs, ret becomes False.

if not ret:
Checks whether the frame was read successfully.

break
Stops the loop if there are no more frames to process.

frame = annotate_frame(frame)
Annotates the current frame with detected signs and student ID.

writer.write(frame)
Saves the annotated frame into the output video file.

cv2.imshow("Video Output", frame)
Displays the annotated frame in a window for real-time visualization.

if cv2.waitKey(1) & 0xFF == ord('q'):
Waits briefly (1 millisecond) for a key press. If the user presses 'q', the loop breaks to end the video early.

cap.release()
Releases the input video capture object, freeing system resources.

writer.release()
Releases the video writer object, ensuring the output file is properly finalized and saved.

cv2.destroyAllWindows()
Closes all OpenCV display windows that were opened during the process.

1.4.9. Ensuring Template Directory Exists and Running the Program:

Purpose:

This part of the code ensures the template image folder is available before executing the main traffic sign detection process. It also defines the main entry point for running the program.

```
# Tạo thư mục nếu chưa có
Tabnine | Edit | Test | Explain | Document                    (module) sign_templates
def ensure_template_folder_exists(path='sign_templates'):
    if not os.path.exists(path):
        os.makedirs(path)

# Gọi chạy
if __name__ == "__main__":
    ensure_template_folder_exists()
    annotate_and_save_video('video1.mp4')
```

def ensure_template_folder_exists(path='sign_templates'):
Defines a helper function that ensures the directory storing sign template images exists. It takes an optional path argument, which defaults to 'sign_templates'.

if not os.path.exists(path):
Checks if the specified folder path exists. If not, the following line creates it.

os.makedirs(path)
Creates the folder (and parent folders if necessary) to store traffic sign template images.

if __name__ == "__main__":
Indicates the entry point of the script. This block only runs when the file is executed directly, not when imported as a module.

ensure_template_folder_exists()
Calls the folder creation function to make sure the 'sign_templates' directory is ready before loading templates.

CHAPTER 2: TASK RESULT

This section presents the output results generated by the implemented image processing techniques in both Task 1 and Task 2. All images are captured and displayed to reflect the effectiveness of the system in detecting, segmenting, and highlighting objects of interest.

1. Video 1: Video-Based Traffic Sign Detection

The following three frames are extracted from the output video generated by Task 1. Each frame shows detected traffic signs bounded by green rectangles. The time difference between each frame is at least 5 seconds, ensuring temporal separation and evaluation at various points in the video sequence.

Figure 1.1 Output the frame at the 5-second mark.

Detection of "No Left Turn" signs

This output frame, captured at the 5th second of the video, shows two correctly detected **"No left turn"** traffic signs. Both signs are highlighted with green rectangles and accurately labeled in red text, indicating their content and dominant color. The system successfully distinguishes multiple instances of the same sign in a single frame and handles real-world conditions such as cluttered backgrounds and partial occlusion.

Figure 1.2 Output the frame at the 6-second mark.

Figure 1.3 Output the frame at the 5-second mark.

Detection of multiple distinct traffic signs

This frame, captured at the 9th second of the video, demonstrates the system's ability to detect and classify **three different traffic signs** simultaneously. The identified signs include:

• "Wrong way" (Red) at the top

• "Keep right" (Blue) in the middle

• "No parking" (Blue and Red) at the bottom

Figure 1.4 Output the frame at the 18-second mark

Detection of directional and warning signs

In this frame captured at the 13th second, the system successfully detects and labels **two prominent traffic signs**:

• "Wrong way" (Red) at the top

• "Keep right" (Blue) below it

Figure 1.5 Output the frame at the 28-second mark

Detection of directional and warning signs

In this frame captured at the 13th second, the system successfully detects and labels **two prominent traffic signs**:

• "Wrong way" (Red) at the top

• "Keep right" (Blue) below it

Figure 1.6 Output the frame at the 43-second mark

Isolated detection of a “No Left Turn” sign

This frame, captured at the 17th second, displays a single clear detection of a **"No left turn"** sign. The sign is placed on a vertical pole near the center of the frame, accurately enclosed by a green bounding box and labeled in red text, reflecting its color and meaning. The result demonstrates the system's ability to isolate and classify a sign even when there is no surrounding clutter or additional signs nearby.

Figure 1.7 Output the frame at the 67-second mark

Detection of complex urban signage

In this frame, taken at the 21st second, the system detects and correctly classifies **three different traffic signs** in a dense urban environment:

• "Wrong way" (Red) at the top-left

• "Keep right" (Blue) below it

• "No stopping and parking" (Red) on the right side of the frame

2. Video 2: Video-Based Traffic Sign Detection:

Figure 2.1. Output the frame at the 20-second mark

Detection of child safety warning signs

This frame, captured at the 25th second, features the correct identification of **two "Children" warning signs**, both displayed in yellow and located on opposite sides of a utility pole. Each sign is accurately outlined with a green rectangle and annotated in red with the label "Children (Yellow)". This result confirms the system's capability to detect warning signs with triangular shapes and yellow color, which are often used in child safety or pedestrian caution zones.

Figure 2.2. Output the frame at the 49-second mark

Detection of a “No Parking” sign

This frame, taken at the 29th second, shows the accurate detection of a **"No parking"** sign. The sign is clearly visible near the center of the image, highlighted with a green bounding box and labeled "No parking (Blue)" in red text. Despite partial shadow and background clutter, the system confidently identifies the circular blue sign, showcasing strong performance in varied lighting conditions.

REFERENCES

[1] OpenCV Documentation. “Image Processing (cv2 module)”, OpenCV-Python Tutorials, https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html

[2] NumPy Documentation. “NumPy Array API Reference”, https://numpy.org/doc/stable/

[3] Matplotlib Documentation. “Pyplot — Matplotlib 3.x Documentation”, https://matplotlib.org/stable/api/pyplot_summary.html

[4] StackOverflow. “How to use cv2.findContours() in OpenCV?”, https://stackoverflow.com/questions/11782147

[5] Gonzalez, R. C., & Woods, R. E. (2018). Digital Image Processing (4th Edition). Pearson.

[6] GeeksForGeeks. “Morphological Operations in Image Processing (Opening, Closing)”, https://www.geeksforgeeks.org/morphological-operations-in-image-processing/

[7] PyImageSearch. “Contours, Bounding Boxes, and Hierarchy in OpenCV”, https://pyimagesearch.com/T. Nguyen and H. Le, "Using predictive analytics to optimize food delivery times," *International Journal of Data Science Applications*, vol. 10, no. 1, pp. 38–45, 2022.